# Indian Institute of Technology Bhubaneswar

**School of Electrical Sciences**

**Autumn Mid-Semester Examination-2022**

| Subject Name: Compiler Design | Subject Code: CS4L001 |
|---|---|

**Date: 15.09.2022**

**Duration: 2 Hours** **Full Marks: 60**

**Answer a new question on a fresh page (strictly) and all parts of a question together.**

**Section A: Multiple choice questions (choose all correct options) – Each question carries 1/2 mark. Write answers in the answer sheet.**

**A1**. In a compiler, keywords of a language are recognized during:

a) parsing of the program
b) the lexical analysis of the program
c) data flow analysis
d) the code generation

**A2**. What is the output of a lexical analyser?

a) A parse tree
b) Intermediate code
c) A list of tokens
d) Machine code

**A3.** A bottom up parser generates:

a) Right most derivation
b) Leftmost derivation
c) Rightmost derivation in reverse
d) Leftmost derivation in reverse

**A4.** Shift-reduce parsers are:

a) Top down Parser
b) Bottom Up parser
c) May be top down or bottom up
d) None of the above

**A5**. An SLR(1) parser for a grammar G can have shift-reduce (S-R) conflicts if and only if the LR(0) parser for G has S-R conflicts.

a) True
b) False
d) None of the above

**A6**. Which of the following describes a handle (as applicable to LR-parsing) appropriately?

a) It is the position in a sentential form where the next shift or reduce operation will occur
b) It is non-terminal whose production will be used for reduction in the next step
c) It is a production that may be used for reduction in a future step along with a position in the sentential form where the next shift or reduce operation will occur
d) It is the production p that will be used for reduction in the next step along with a position in the sentential form where the right hand side of the production may be found

**A7**. Grammar is ambiguous if

a) it has 2 parse trees
b) it has two left most derivations
c) it has two right most derivations
d) all of the above

**A8**. Which of the following suffices to convert an arbitrary CFG to an **LL(1)** grammar?

a) Removing left recursion only
b) Factoring the grammar alone
c) Factoring & left recursion removal
d) None of the above

**Section B: Answer the following questions briefly – 3+2+2+2+3+2+2+3= 17.**

**B1**. What is a compiler? List the various phases of a compiler, mentioning the purpose of each phase briefly.
**B2**. What are the benefits of intermediate code generation?
**B3**. Differentiate tokens, patterns, lexeme. Explain via an example.
**B4**. What is semantic analysis? What does semantic analysis involve?
**B5**. What is meant by viable prefixes? Explain via an example..
**B6**. Briefly describe how an LR parser works.
**B7**. Consider the grammar with non-terminals **N = {S, C, S1}**, terminals **T= {a, b, i, t, e}**, with S as the start symbol, and the following set of rules: **S --> iCtSS1|a  S1 --> eS|ϵ  C --> b**
Is the grammar left-recursive? Is it ambiguous? Justify your answer.

**Section C: Lexical analysis— 2+3+2+5 = 12 marks**

**C1.** A regular expression of a language over *{a, b}* is **(a|(a*b))**. Design a DFA for the language.

**C2.** A simple 'maximal-length' tokenizer is designed for the language of (C1). The input to the scanner is $a^{50}$<eof> (50 a's followed by end-of-file). How many character-read and roll-backs are required to tokenize the considered input.

**C3.** Given the following tokens and their associated regular expressions, show what output is produced when this lex scanner is run over the following string: **bbbaacacbabcacabbccc**

```
%%
aa*(b|c)*a     { printf(" T1 "); }
bb*(a|c)*b     { printf(" T2 "); }
cc*(b|a)*c     { printf(" T3 "); }
.|\n           { printf(" ? ");  }
%%
```

4. In the Ada Programming language an integer constant contains one or more digits, but it may also contain embedded underscores. Any underscores must be preceded and followed by at least one digit, which implies that a number may not begin or end with an underscore, and numbers may not contain two or more adjacent underscores. Underscores have no significance other than readability.
*Examples*: 0, 17, 017, 0_01_7, 1_048_576.

Integers may also be written in bases other than 10 by preceding the number with the base and a # character. For example, all of these constants represent the decimal number 15231: 15231, 10#15231, 16#3B7F, 16#03B_7F, 2#00111011_0111111, and 8#35577.

We will simplify/ignore some constraints that the language imposes, and finally consider the following constraints/requirements for this question:

- The *base of a number* (preceding the # sign) must be a number that is at least 2 and at most 16, and is written in decimal.
- The *digits in a number* consist of the decimal digits 0 through 9, and the letters A, B, C, D, E, F, written only in upper case.
- If no base is present, the digits may only be 0 through 9.

**(a) Give a regular definition for Ada integer constants as described above.**

**(b) Draw a DFA (Deterministic Finite Automata) that recognizes Ada integer constants as described in the problem and generated by the regular definition in your answer to part (a).**

**Section D: Top-down Parsing — 3+3+3+3= 12 marks.**

Consider the following grammar with non-terminals {**S, B, A**}, start symbol **S**, terminal symbols { **n**, +, ×}:

S -> n B

B -> n B A B | ϵ

A -> + | ×

**D1**. What are the conditions for a grammar to be **LL(1)**? Is the above grammar **LL(1)**? Make changes to the above grammar (if necessary) to make it suitable for LL(1) parsing.

**D2**. Briefly describe the rules for computing *FIRST* and *FOLLOW*. Compute *FIRST* and *FOLLOW* sets for each non-terminal.

**D3**. Compute **LL(1)** parse-table.

**D4**. Show the actions of the LL(1) parser given the input string: ***n n*** + ***n*** ×

**Section E: Bottom-up Parsing (LR(0), SLR): 2.5+5+2.5= 10 marks.**

Consider the CFG fragment with non-terminal symbols **{D, T, L}**, with start symbol **S**, terminal symbols { **id**, :, ;, ,, **var**, **integer**, **real** } and the productions **P** listed below.

D → var L : T ; $

T → integer

T → real

L → L , id

L → id

**(E1)** Is this grammar suitable to be parsed using a predictive **LL** parsing algorithm? Why?

**(E2)** Compute the DFA that recognizes the **LR(0)** sets of items for this grammar and construct the corresponding **LR(0)** parsing table. Comment on the nature of the conflicts, if any.

**(E3)** Typically the **LR(0)** table construction algorithm leads to tables that are very 'dense'. Construct the **SLR** parsing table by indicating the *FOLLOW* sets used to trim the entries on the table.

**Section F: Bottom-up Parsing (LR(0), SLR(1)): 5 marks**

Consider the following grammar with the set of non-terminals **{S, A, B}**, start symbol **S**, and terminal symbols **{a, b, c, d}**. Answer whether the grammar is **LR(0)**. Is the grammar **SLR(1)**? Justify your answer.

| |
|---|
| (1) **S → Aa** |
| (2) **S → bAc** |
| (3) **S → Bc** |
| (4) **S → bBa** |
| (5) **A → d** |
| (6) **B → d** |

**NOTE**: Augment the grammar with the production **S' → S$**